

random_forest

RandomForestConfig

```
RandomForestConfig(num_trees: int = 100, max_depth: int = 20, split_criteria: str = 'One vs others', min_instance_split: int = 2, in_bag: float = 0.65, features_per_split: int = 0, undersampling: float | None = None, oversampling: float | None = None, seed: int | None = None, ignored_fields: set[str] | None = None, schema: Schema | None = None)
```

Bases: [TrainingConfig](#)

The Feedzai Random Forest configuration object used for model training.

Parameters:

Name	Type	Description	Default
<code>num_trees</code>	int	The number of trees, defaults to 100.	100
<code>max_depth</code>	int	The depth of each tree, defaults to 20.	20
<code>split_criteria</code>	str	Split criteria for categorical values, can be "All vs all", or "One vs others". Defaults to "One vs others".	'One vs others'
<code>min_instance_split</code>	int	The minimum number of split instances, defaults to 2.	2
<code>in_bag</code>	float	The ratio used for sampling data into the training, defaults to 0.65.	0.65
<code>features_per_split</code>	int	The number of features per split, defaults to 0 which represents sqrt(N).	0
<code>undersampling</code>	float	The value for undersampling. Only this or oversampling can be used.	None
<code>oversampling</code>	float	The value for oversampling. Only this or undersampling can be used.	None
<code>seed</code>	int	The seed value.	None
	set of str	Set of names corresponding to the fields to be ignored when training a model.	None

Name	Type	Description	Default
ignored_fields ↗			
schema ↗	Schema	Pass a schema to enable conversion between descscs and internal names. By not passing a schema, ds-api will assume field names are references to internal Pulse names, not the ones in the UI.	None

Examples:

```
>>> model_config = RandomForestConfig(
...     num_trees=200,
...     in_bag=0.5,
...     undersampling=0.98
... )
>>> model_config.num_trees # gets the num_trees: 200
>>> model_config.num_trees = 100 # sets the num_trees to 100
>>> model = model_project.train(
...     "model_name",
...     datasource,
...     "target_field",
...     model_config
... )
>>> model.waitUntilModelTrained()
>>> config_from_model = model.get_config()
```

features_per_split property writable [↗](#)

features_per_split

ignored_fields property writable [↗](#)

ignored_fields

in_bag property writable [↗](#)

in_bag

max_depth property writable [↗](#)

max_depth

min_instance_split property writable [↗](#)

min_instance_split

num_trees property writable [↗](#)

num_trees

oversampling property writable [↗](#)

oversampling

seed property writable [↗](#)

seed

split_criteria property writable [¶](#)

split_criteria

subclasses class-attribute instance-attribute [¶](#)

subclasses: List[[JavaWrapper](#)] = []

undersampling property writable [¶](#)

undersampling

auto_cast staticmethod [¶](#)

auto_cast([config](#): [JavaWrapper](#)) -> [RandomForestConfig](#)

Creates a RandomForestConfig from the java object.

Parameters:

Name	Type	Description	Default
config ¶	JavaWrapper	Java configuration object.	<i>required</i>

Returns:

Type	Description
RandomForestConfig	Configuration object according to its python representation.